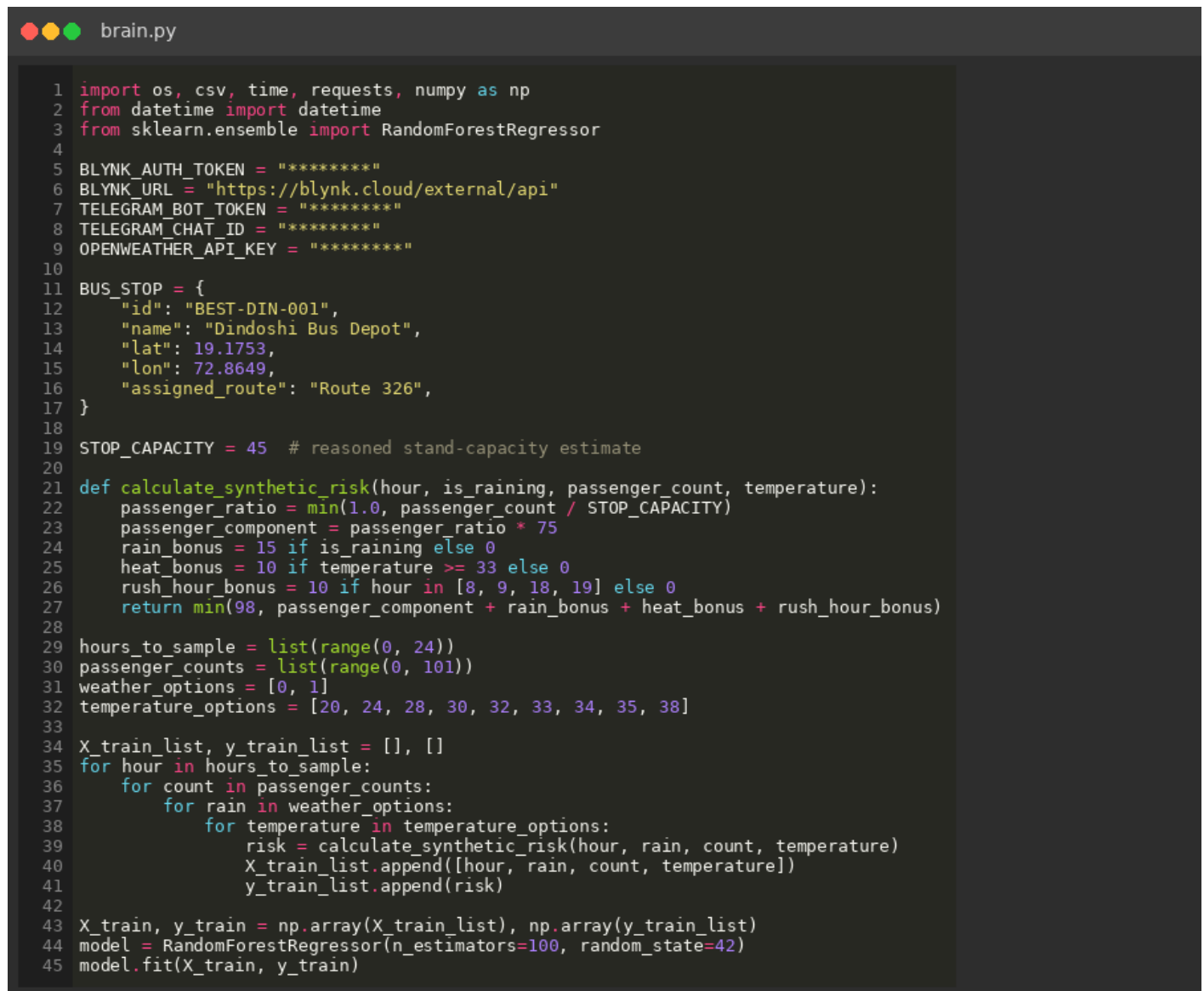


1. BACKEND CORE LOGIC

This code builds a 43,632-sample synthetic grid and trains RandomForestRegressor(n_estimators=100, random_state=42) on hour, rain, passenger count and temperature using the locked risk formula.



```

1 import os, csv, time, requests, numpy as np
2 from datetime import datetime
3 from sklearn.ensemble import RandomForestRegressor
4
5 BLYNK_AUTH_TOKEN = "*****"
6 BLYNK_URL = "https://blynk.cloud/external/api"
7 TELEGRAM_BOT_TOKEN = "*****"
8 TELEGRAM_CHAT_ID = "*****"
9 OPENWEATHER_API_KEY = "*****"
10
11 BUS_STOP = {
12     "id": "BEST-DIN-001",
13     "name": "Dindoshi Bus Depot",
14     "lat": 19.1753,
15     "lon": 72.8649,
16     "assigned_route": "Route 326",
17 }
18
19 STOP_CAPACITY = 45 # reasoned stand-capacity estimate
20
21 def calculate_synthetic_risk(hour, is_raining, passenger_count, temperature):
22     passenger_ratio = min(1.0, passenger_count / STOP_CAPACITY)
23     passenger_component = passenger_ratio * 75
24     rain_bonus = 15 if is_raining else 0
25     heat_bonus = 10 if temperature >= 33 else 0
26     rush_hour_bonus = 10 if hour in [8, 9, 18, 19] else 0
27     return min(98, passenger_component + rain_bonus + heat_bonus + rush_hour_bonus)
28
29 hours_to_sample = list(range(0, 24))
30 passenger_counts = list(range(0, 101))
31 weather_options = [0, 1]
32 temperature_options = [20, 24, 28, 30, 32, 33, 34, 35, 38]
33
34 X_train_list, y_train_list = [], []
35 for hour in hours_to_sample:
36     for count in passenger_counts:
37         for rain in weather_options:
38             for temperature in temperature_options:
39                 risk = calculate_synthetic_risk(hour, rain, count, temperature)
40                 X_train_list.append([hour, rain, count, temperature])
41                 y_train_list.append(risk)
42
43 X_train, y_train = np.array(X_train_list), np.array(y_train_list)
44 model = RandomForestRegressor(n_estimators=100, random_state=42)
45 model.fit(X_train, y_train)

```

Image.1: Backend Core Logic

2. LIVE MONITORING AND CSV LOG

This code fetches Dindoshi weather from OpenWeatherMap, reads V1 and V4 from Blynk, predicts risk, classifies NORMAL / WARNING / CRITICAL, and appends one row to bus_stop_log.csv every five seconds.

```

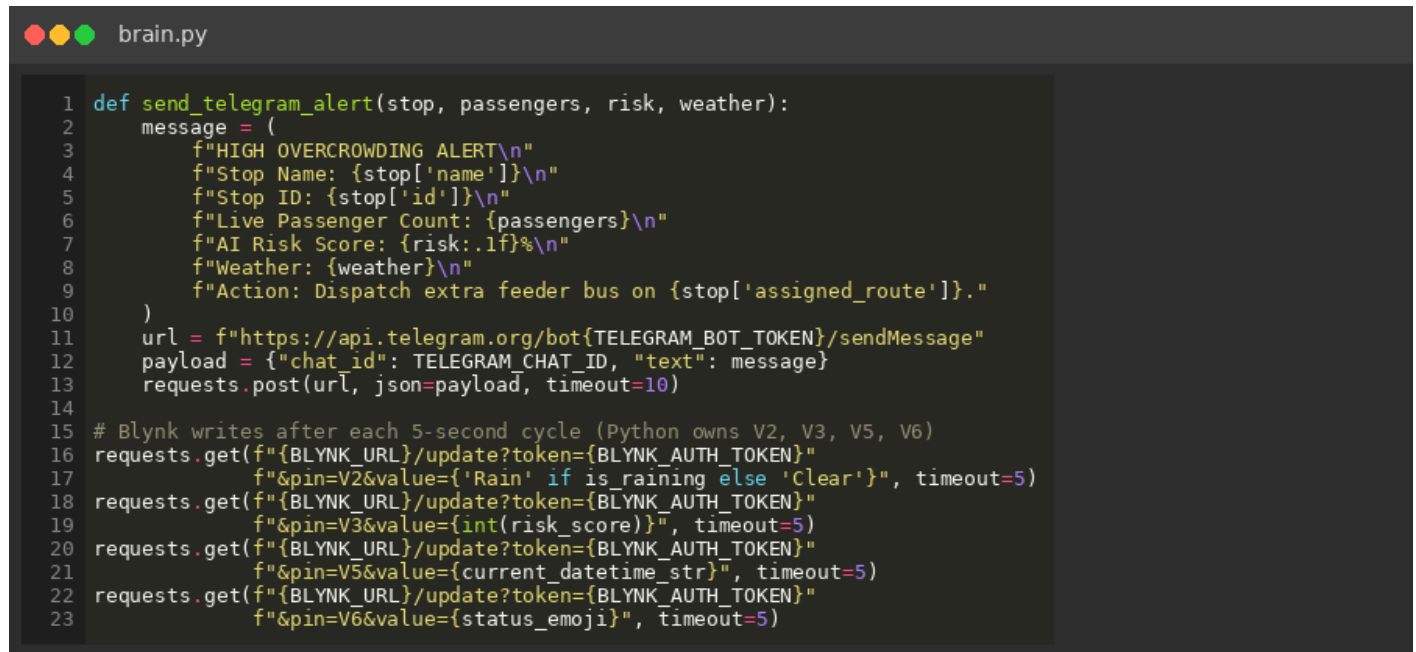
1 def get_weather():
2     try:
3         url = (f"https://api.openweathermap.org/data/2.5/weather"
4             f"?lat={BUS_STOP['lat']}&lon={BUS_STOP['lon']}"
5             f"&appid={OPENWEATHER_API_KEY}&units=metric")
6         data = requests.get(url, timeout=10).json()
7         main = data["weather"][0]["main"]
8         if main in ("Rain", "Drizzle", "Thunderstorm", "Shower"):
9             return "Rain"
10        return "Clear"
11    except Exception:
12        return "Clear" # fallback: Clear, 28.0 C used if DHT22 missing
13
14 def get_blynk_value(pin):
15     res = requests.get(
16         f"{BLYNK_URL}/get?token={BLYNK_AUTH_TOKEN}&pin={pin}", timeout=5)
17     val = res.text.strip('[]' '\n')
18     if "error" in val.lower() or not val:
19         return None
20     return float(val)
21
22 if not os.path.exists("bus_stop_log.csv"):
23     with open("bus_stop_log.csv", "w", newline="", encoding="utf-8") as f:
24         csv.writer(f).writerow(
25             ["timestamp", "passengers", "temp", "weather", "risk", "status"])
26
27 while True:
28     weather_status = get_weather()
29     is_raining = 1 if weather_status == "Rain" else 0
30     passenger_count = int(float(get_blynk_value("V1") or 0))
31     temp_value = float(get_blynk_value("V4") or 28.0)
32     features = np.array([[datetime.now().hour, is_raining,
33         passenger_count, temp_value]])
34     risk_score = model.predict(features)[0]
35
36     # <=40 NORMAL | 41-75 WARNING | >75 CRITICAL (exactly 75 = WARNING)
37     if risk_score > 75:
38         csv_status = "CRITICAL"
39         if time.time() - last_telegram_alert_time > 60:
40             send_telegram_alert(BUS_STOP, passenger_count, risk_score, weather_status)
41             last_telegram_alert_time = time.time()
42     elif risk_score > 40:
43         csv_status = "WARNING"
44     else:
45         csv_status = "NORMAL"
46
47     with open("bus_stop_log.csv", "a", newline="", encoding="utf-8") as f:
48         csv.writer(f).writerow([
49             datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
50             passenger_count, f"{temp_value:.1f}", weather_status,
51             f"{risk_score:.1f}", csv_status])
52     time.sleep(5)

```

Image.2: Live Loop and CSV Logging

3. TELEGRAM ALERT AND BLYNK WRITE

This code sends a Telegram overcrowding alert when risk is above 75, subject to a 60-second cooldown, and writes weather, risk, time and status back to Blynk pins V2, V3, V5 and V6.



```

1 def send_telegram_alert(stop, passengers, risk, weather):
2     message = (
3         f"HIGH OVERCROWDING ALERT\n"
4         f"Stop Name: {stop['name']}\n"
5         f"Stop ID: {stop['id']}\n"
6         f"Live Passenger Count: {passengers}\n"
7         f"AI Risk Score: {risk:.1f}%\n"
8         f"Weather: {weather}\n"
9         f"Action: Dispatch extra feeder bus on {stop['assigned_route']}."
10    )
11    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
12    payload = {"chat_id": TELEGRAM_CHAT_ID, "text": message}
13    requests.post(url, json=payload, timeout=10)
14
15 # Blynk writes after each 5-second cycle (Python owns V2, V3, V5, V6)
16 requests.get(f"{BLYNK_URL}/update?token={BLYNK_AUTH_TOKEN}"
17             f"&pin=V2&value={'Rain' if is_raining else 'Clear'}", timeout=5)
18 requests.get(f"{BLYNK_URL}/update?token={BLYNK_AUTH_TOKEN}"
19             f"&pin=V3&value={int(risk_score)}", timeout=5)
20 requests.get(f"{BLYNK_URL}/update?token={BLYNK_AUTH_TOKEN}"
21             f"&pin=V5&value={current_datetime_str}", timeout=5)
22 requests.get(f"{BLYNK_URL}/update?token={BLYNK_AUTH_TOKEN}"
23             f"&pin=V6&value={status_emoji}", timeout=5)

```

Image.3: Telegram and Blynk Output

4. ESP32 FIRMWARE LOGIC

This firmware confirms a passenger crossing after two consecutive VL53L0X readings below 400 mm, applies V7 as $\max(0, \text{count}-30)$, drives the LEDs from the Python risk on V3, and publishes V1, V4 and V8.

```

smart_bus_stop.ino

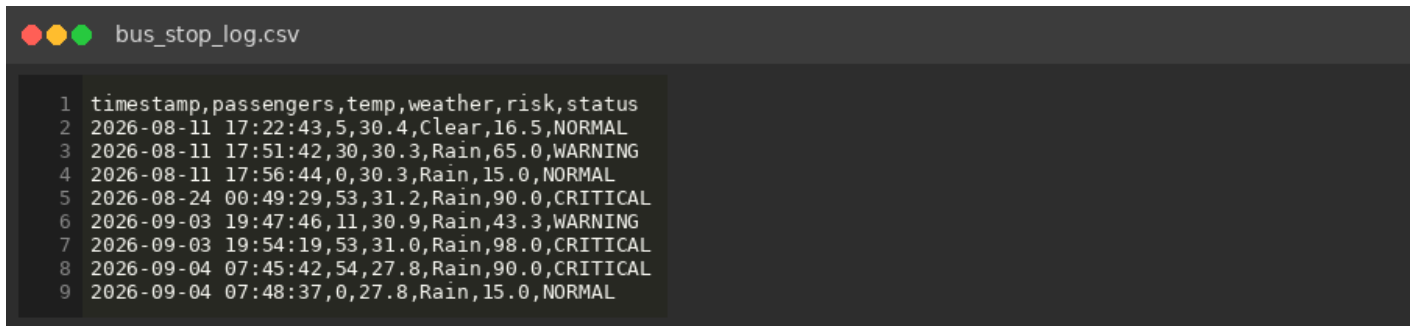
1 #define BLYNK_AUTH_TOKEN "*****"
2 char ssid[] = "*****";
3 char pass[] = "*****";
4
5 #define DHTPIN 4
6 #define RED_LED 25
7 #define GREEN_LED 26
8 const int CONFIRM_THRESHOLD = 2;
9 const int DETECTION_THRESHOLD_MM = 400;
10 const unsigned long TOF_CHECK_INTERVAL = 150;
11
12 bool isPersonDetected() {
13     VL53L0X_RangingMeasurementData_t measure;
14     lox.rangingTest(&measure, false);
15     return (measure.RangeStatus != 4 &&
16           measure.RangeMilliMeter < DETECTION_THRESHOLD_MM);
17 }
18
19 void checkPassengerCrossing() {
20     bool detected = isPersonDetected();
21     if (detected) {
22         consecutiveDetections++;
23         if (consecutiveDetections >= CONFIRM_THRESHOLD && !personInFrame) {
24             passengerCount++;
25             personInFrame = true;
26             lastCrossingTime = millis();
27         }
28     } else {
29         consecutiveDetections = 0;
30         personInFrame = false;
31     }
32 }
33
34 // V7 Bus Left: subtract 30, never auto-reset to zero
35 BLYNK_WRITE(V7) {
36     if (param.asInt() == 1) {
37         passengerCount = max(0, passengerCount - 30);
38     }
39 }
40
41 // V3 from Python: red LED only when risk > 75 (CRITICAL)
42 BLYNK_WRITE(V3) {
43     pythonRiskScore = param.asInt();
44     if (pythonRiskScore > 75) {
45         digitalWrite(RED_LED, HIGH);
46         digitalWrite(GREEN_LED, LOW);
47     } else {
48         digitalWrite(RED_LED, LOW);
49         digitalWrite(GREEN_LED, HIGH);
50     }
51 }
52
53 void sendDashboardData() {
54     Blynk.virtualWrite(V1, passengerCount);
55     Blynk.virtualWrite(V4, dht.readTemperature());
56     Blynk.virtualWrite(V8, (millis() - lastCrossingTime) / 60000);
57 }

```

Image.4: ESP32 Crossing, V7 and LED Logic

5. CSV SENSOR LOG

This extract of bus_stop_log.csv shows live cycles at Dindoshi, including 65.0% WARNING at 30 passengers with rain and 90.0% CRITICAL at full-capacity rain cases.

A screenshot of a code editor window titled 'bus_stop_log.csv'. The window displays a CSV file with 9 lines of data. The first line is a header with columns: timestamp, passengers, temp, weather, risk, status. The subsequent lines contain data for various dates and times, including passenger counts, temperature, weather conditions, risk percentages, and status labels (NORMAL, WARNING, CRITICAL).

```
1 timestamp,passengers,temp,weather,risk,status
2 2026-08-11 17:22:43,5,30.4,Clear,16.5,NORMAL
3 2026-08-11 17:51:42,30,30.3,Rain,65.0,WARNING
4 2026-08-11 17:56:44,0,30.3,Rain,15.0,NORMAL
5 2026-08-24 00:49:29,53,31.2,Rain,90.0,CRITICAL
6 2026-09-03 19:47:46,11,30.9,Rain,43.3,WARNING
7 2026-09-03 19:54:19,53,31.0,Rain,98.0,CRITICAL
8 2026-09-04 07:45:42,54,27.8,Rain,90.0,CRITICAL
9 2026-09-04 07:48:37,0,27.8,Rain,15.0,NORMAL
```

Image.5: CSV Log Extract